CS780 Capstone Project:

OBELIX: the Warehouse Robot

The best way to learn Reinforcement Learning is to get your hands dirty by actually doing it.

Table of Contents

Table of Contents **2**
1. Your First Job in the Industry 3
2. Task Description 4
    2.1 Setting 4
    2.2 Difficulty Levels 5
    2.3 Competition Format 6
4. Environment and Codebase 7
    4.1 Environment Description 7
5. Evaluation 9
    5.1 Metric 9
    5.2 Development/Test Phase Evaluation 9
    5.3 LLM Usage Policy 10
6. Getting Started: Implementation and Code 10
    6.1 How to get started? 11
    6.2 Starter Code 11
        6.2.1 Double Deep Q-Network (DDQN) 12
7. Codabench Leaderboard 12
    7.1 Competition Link 12
    7.2 Registration 13
    7.3 Submission Policy 13
    7.4 Environment & Dependency Requirements 13
8. Evaluation / Grading Criteria 14
9. Submission Details: Report and Code 14
10. Timeline 15
    10.1 Phase 1 (Level 1 Dev Phase) 15
    10.2 Phase 2 (Level 2 Dev Phase) 15
    10.3 Phase 3 (Level 3 Dev Phase) 15
    10.4 Phase 4 (Test Phase) 15
11. Possible Ideas 15
    11.1 Background Tutorials 15
    11.2 Tabular / Value-based Methods 16
    11.3 Function Approximation and Deep RL 16
    11.4 Evolutionary Strategies & Heuristics 16
    11.5 What approach should I take? 16
12. Lost? Whom to contact? 17
13. References 17

1. Your First Job in the Industry

You have recently graduated and joined a robotics startup that develops warehouse robots for large e-commerce companies. The company has been building delivery and warehouse robots that need to interact (e.g., carry, move, etc.) with objects in dynamic environments. The CTO wants to move beyond hand-coded behaviors (which are brittle and hard to scale) and leverage modern reinforcement learning to make robots learn complex manipulation tasks from trial-and-error feedback. Your task is to design an RL-based policy that maps sensor observations to robot actions.

CTO got to know that you have done CS780: Deep Reinforcement Learning course from IITK; she decided to hand over this project to you since you are among the few ones (due to CS780 background) who know DRL inside out. The CTO assigns you the task of developing an autonomous controller for a mobile robot (called as **OBELIX**) that can locate, attach to, and push objects (like boxes) to desired regions reliably even in partially observable or changing conditions.

Meanwhile, someone else in the company also claims that they can hand-craft or evolve a better controller. To make it fair and exciting, the CTO organizes an internal competition: design the best algorithm for the **OBELIX** robot in a simulated arena. The robot draws inspiration from the classic 1991 paper by Sridhar Mahadevan and Jonathan Connell [1], one of the earliest real demonstrations of reinforcement learning on a physical robot.

In this competition, you will implement a RL algorithm that takes binary or discrete sensor readings as input and outputs one of five discrete actions. The goal is to make the robot find the box, attach/push it, and drive it to the boundary (goal) as quickly and reliably as possible across randomized starting positions and increasing difficulty levels.

This is an **individual competition**, not a group project. There will be a public leaderboard on Codabench where your solution will be ranked. **Do not share code or models with classmates** unless you want to share grades and career too! Code similarity checks and viva will be strict : copying leads to **F grade, no questions asked**!

Everyone must participate actively: aim for multiple submissions during the development phase to show your experimentation efforts.

2. Task Description

2.1 Setting

You are given a robot equipped with discrete sensors and a fixed set of 5 actions: rotate right 45°, rotate right 22°, move forward, rotate left 22°, and rotate left 45° (details mentioned in section 4). The objective is to design an algorithm (policy) that finds, pushes and unwedges (at the goal state) the grey box from the arena while using the minimum possible number of steps within an episode.

At the start of each episode, a grey box is generated at a random location inside the arena. To remove the box, the robot must navigate toward it using its sensor observations, attach to it, and push it until the particle reaches the arena boundary, which defines the goal region. The robot automatically attaches to the grey box upon contact and automatically unwedges/detaches once the box reaches the boundary. The box is considered removed only after it is successfully pushed outside the boundary.

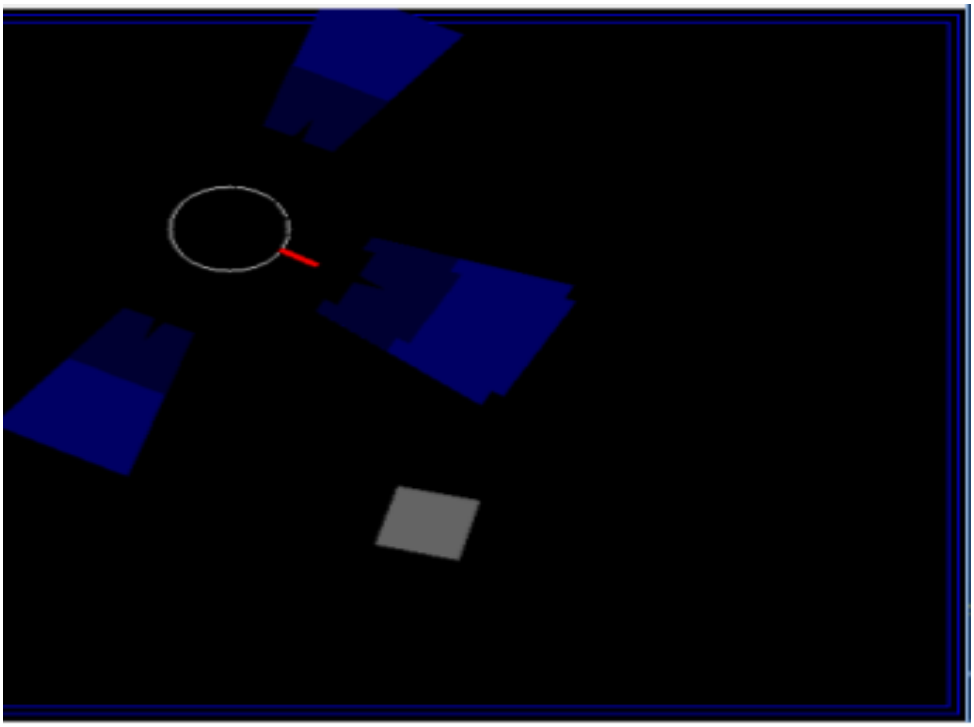
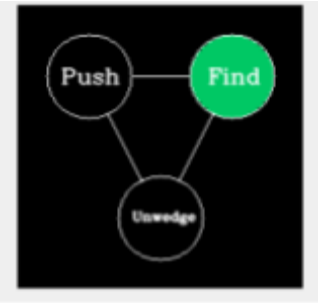


Image of the robot and environment. The sensors and their range is visible along with the infrared front sensor. The blue boundary is the goal state.

The robot observes the environment using sonar-like sensors that indicate whether the box is near or far. The figure above shows the robot, arena, grey box, boundary area sensor range, and directions. There are a total of 8 sensors around the robot, 4 towards the front and 2 on either side. The sensors provide a bit-based output that is set to 1 if the sensor detects the box within its region. If the box is in the light blue area, the near bit is set to 1; if it's in the dark blue area, the far bit is set to 1. The robot also has an infrared sensor (IR sensor) (represented as a red line on the front), which is set to 1 if the grey box is sensed by it. Additionally, the robot also senses if the box is attached to it. This makes the 18 bits of sensory information that the robot receives for each observation.

For human readability, these are mentioned as 3 distinct states. First, the robot “Finds” the grey box, then attaches to it and “Pushes” the box, and finally “Unwedeges” the box at the boundary. Note, these states are **conceptual for humans** and are **not directly given to the robot**. The robot must infer them from sensor readings.



Discrete states describing the reward behaviour. This is not the actual state visible to the agent.

Note that, for the robot, the environment is partially observable, since it only sees within the range allowed by its sensors. If the box is far from the robot, it may not even know exactly where the box is. It would first need to explore a bit to get within the box's range. More specifically, the robot does not know the full state of the environment. It only receives **local sensor readings (observations)**, making this a **Partially Observable Markov Decision Process (POMDP)**.

The agent is evaluated based on the cumulative reward obtained at the end of the episode. At each step, the robot receives a positive reward if any sensor bit is set to 1 and it moves forward. Details on the reward obtained at each step are mentioned in section on Environment and Codebase details. Episodes terminate when either the box is unwedged or a maximum step limit (episode horizon) is reached.

2.2 Difficulty Levels

There are 3 difficulty levels across which the robot is evaluated.

Level 1 – Static Box

- Box remains stationary
- Robot must locate and push it to boundary

Level 2 – Blinking Box

- Box randomly **appears and disappears**
- Robot **cannot attach when box is invisible**
- Once attached → box stops blinking

Level 3 – Moving + Blinking Box

- Box moves with constant velocity
- Box also blinks (like Level 2)
- Movement stops once robot attaches

All these settings increase the difficulty by introducing partial observability and non-stationary dynamics.

Additionally, in each of the levels, a **vertical wall with a small opening** may appear.

- The robot cannot push through the wall.
- If the robot pushes the box into the wall, it receives a negative **reward**.
- The robot must navigate through the opening to reach the boundary.

2.3 Competition Format

There will be essentially two phases of the competition: Development Phase and Testing Phase.

Further, the development phase is sub-divided into three sub-phases, each corresponding to three levels above. The final test phase will have combinations of all the three levels.

For each (sub-)phase you will have to submit the learned RL policy algorithm (in the form of a python code file) on Codabench (details provided later in the document). The trained agent with a fixed policy is evaluated on all three difficulty levels for 10 runs/episodes with fixed hidden seeds on Codabench. Cumulative reward is captured across each episode and averaged across 10 episodes to get the mean cumulative reward for each difficulty level. This is displayed on the leaderboard along with an average reward score across all the difficulty levels. Note that the evaluation environment uses **hidden random seeds**, meaning you cannot overfit to specific scenarios.

Check out the Competition timeline in here

4. Environment and Codebase

The simulation is in this GitHub repo: https://github.com/Exploration-Lab/CS780-OBELIX

```
git clone https://github.com/Exploration-Lab/CS780-OBELIX
cd CS780-OBELIX
pip install -r requirements.txt
```

Please use the instructions provided in the README.md file present in the repo.

- Run `manual_play.py` to drive the robot manually (keys: w=forward, a/q=left turns, d/e=right turns) and observe sensor feedback and reward in the terminal and the states in another box (as shown in the images above). Note you can run the environment on your laptop (CPU), GPU is not required.

4.1 Environment Description

- **Observations (partial state):** A vector of 18-bit readings (16 bits from the sonar sensors, a bit for the infrared sensor and a box attachment bit)
  - Example observation:

```
obs = [0,1,0,0,1,0,0,1, ...]  # length 18
```

Sensor Type	Count	Description
Sonar sensors (**near**)	8	Detect box in **near** region
Sonar sensors (**far**)	8	Detect box in **far** region
Infrared sensor	1	Detect box directly ahead
Attachment sensor	1	Indicates robot is attached to grey box

- **Actions:** Discrete 5: "L45" (left 45°), "L22" (left 22.5°), "FW" (forward), "R22", "R45".

Action	Description
L45	Rotate left 45°
L22	Rotate left 22.5°
FW	Move forward
R22	Rotate right 22.5°
R45	Rotate right 45°

- **Rewards:** The agent receives a reward at each time step according to the observation table below

Observation	Reward
left/right far/near sensor bit is set to 1 (this reward is incurred only once during an episode)	+1
Forward sensor **far** bit is set to 1 (this reward is incurred only once during an episode)	+2
Forward sensor **near** bit is set to 1 (this reward is incurred only once during an episode)	+3
Infrared sensor bit is set to 1 (this reward is incurred only once during an episode)	+5
• Robot is stuck against wall • Robot is pushing grey box into wall • Robot is not able to move forward at the arena boundary without a grey box	-200
None of the sensor bits are set to 1	-1
On enabling “push” state. Initial attachment to grey box	+100

Each step in “push” state	-1
Grey box at boundary (Successful termination)	+2000

- **Difficulty levels** (via flags in `evaluate.py`):
  - Static box
  - Blinking/appearing-disappearing box
  - Moving + blinking box (adjust --box_speed N)
  - Optional: --wall_obstacles for enabling the wall obstacle

- **Scoring:** Cumulative reward over fixed max_steps (2000 steps); terminal bonus on success; mean/std over multiple runs/seeds.

You need to implement policy algorithm in python and store it in file named as: `agent_template.py`, the file should contain the following function with the definition given below (it may have other helper functions), more details in section on Submission Details:

```
def policy(obs) -> str:
# return one of: "L45", "L22", "FW", "R22", "R45"
```

In above “obs” is the input observations

Note: Current push is "attach and stick" (simplified); real physics pushing not fully simulated.

NOTE the submitted RL policy algorithm file should have the exactly the same function definition as given above, as your algorithm will be automatically evaluated using a script.

5. Evaluation

5.1 Metric

We use mean cumulative reward (higher = better) as the evaluation metric. The leaderboard shows the mean cumulative reward across 10 runs with and without the wall obstacle during the development phase. The leaderboard will show the mean cumulative reward across 10 runs with and without the wall obstacle for all 3 difficulty levels during the test phase.

5.2 Development/Test Phase Evaluation

During each development sub-phase, the agent is evaluated for the respective difficulty level with and without the wall obstacle for 10 runs with a step horizon of 2000 steps. During the final test phase, the submitted agent will be evaluated across all three difficulty levels with and without the wall obstacle.

NOTE: You are required to submit during each of the development sub-phases and the test phase. Each of the (sub-)phases has a specific weightage.

5.3 LLM Usage Policy

It is OK to use LLM or other sources (websites, blogs, books, etc) for:

- Coding ancillary parts of the algorithm such as plotting graphs, etc.
- To develop understanding of the algorithm in the case it is not clear to you

It is NOT OK to use LLM or other sources (websites, blogs, books, etc) for:

- Writing the code for the main parts of the algorithm especially given that I have already provided pseudo-code in the lectures. (You can use pseudo code as it is from the lecture slides)
- For doing hyper-parameter tuning of the algorithm.
- For doing analysis of the results and drawing conclusions about it

Since RL is a specialized domain, it is very straightforward to detect when someone (especially a newbie) is taking code from LLM or other sources. In case you are found to do so (even for a small part), there will be no questions asked direct zero will be awarded to you along with further penalties which will affect your final grade. In short, you are expected to implement the algorithms **based on course lectures and pseudocode**.

Be a smart agent (remember Honesty is the best Policy), it is ok to loose a few marks than getting a zero + penalties.

Remember that grading is relative and you are not the only one who is finding it tough!

6. Getting Started: Implementation and Code

Use Python 3 (PyTorch to be used for deep methods). You can use numpy, torch, etc. Code must be well-documented: explain architecture, hyperparameters, and experiments. Cite inspirations/papers. No LLM code generation allowed during competition.

6.1 How to get started?

While there may be several ways to approach the problem, here are some suggestive steps from our side to get started:

1. Clone the GitHub repo: https://github.com/Exploration-Lab/CS780-OBELIX
2. Explore the environment manually using the `manual_play.py` script. There are a bunch of options that you can use, including the `--wall_obstacle` option to enable the static wall. The robot rotates and moves using the keyboard keys, specifically
   - w: “FW” (forward)
   - a: “L22” (rotate left 22°) | q: “L45” (rotate left 45°)
   - d: “R22” (rotate right 22°) | e: “R45” (rotate right 45°)
3. Understand the observation vector and the reward calculation from the observation vector. This will help you identify your trained agents' shortcomings and learn how to discuss your policies qualitatively.
4. Implement your agent using any of the policy control or value algorithms using the `policy(obs)` function in the `agent_template.py` file. The policy function must return the required action at that step, specifically one of **L45, L22, FW, R22, R45**
5. Train your agent using the environment specified in `obelix.py`
6. Evaluate your agent locally using `python evaluate.py --level 0` for the easiest level. Use `--level 2` for the second level and `--level 3` for the hardest level
7. Try out different RL algorithms locally and make submissions to Codabench.

6.2 Starter Code

To get you started, we have prepared a small toy algorithm for you to get started. We have provided a simple training implementation of Double Deep Q-Network (DDQN) in the starter code to help you understand how OBELIX could be trained in the environment.

For submission on codabench, you only need to include:

`agent.py + weights.pth`

inside the `submission.zip` file.

6.2.1 Double Deep Q-Network (DDQN)

The starter training code is based on Double Deep Q-Network, which is an improved version of Deep Q-Network (DQN).

One of the main problems with the original DQN algorithm is overestimation of action values. In vanilla DQN, the same network is used for two tasks:

1. Selecting the best action in the next state
2. Evaluating the value of that selected action

When Q-value estimates are noisy (which is common during early training), this can create optimistic bias, where the agent incorrectly believes that certain actions are much better than they actually are. This often leads to unstable learning.

DDQN addresses this problem by separating these two responsibilities. Instead of using the same network for both tasks, DDQN uses two networks:

- Online (main) network → selects the best action
- Target network → evaluates the value of that action

This simple modification significantly reduces overestimation bias and results in more stable learning.

For more details about Double DQN, please refer to the original paper: Deep Reinforcement Learning with Double Q-learning
https://arxiv.org/pdf/1509.06461

We will highly recommend that you register for codabench and make one submission on it using the starter code, this will help you to get accustomed to the entire process.

Please check the starter code and the submission format here. Also, for environment setup, please refer to the Enviornment Setup.

7. Codabench Leaderboard

7.1 Competition Link

The competition is hosted on Codabench:
https://www.codabench.org/competitions/14572/?secret_key=40fb7a5d-1364-44c7-a547-055b16b12040

7.2 Registration

To take part in the competition, you need to register on Codabench. For this, you need to create an account in the following format:

`FirstLetterOfName_Rollno` (e.g., `a_123456` for Ashutosh Modi).

We suggest you create an account (username in the format above) and register. Make one submission using the starter code to get used to the entire process.

NOTE if you do not create an account with the above mentioned username format, you will not be registered in the competition.

7.3 Submission Policy

- Minimum **4 submissions per week** in each development sub-phase is ***mandatory***.
- Minimum **4 submissions in the final test phase** is ***mandatory***.
- The last valid submission before the deadline is your final entry.
- Please do not rush for last-minute submissions, as Codabench takes around 15–20 minutes to evaluate each submission. Since many students will be participating, the evaluation time may increase due to higher load on Codabench. Therefore, it is advised not to wait until the last moment. Please try to make your submissions in the initial days of the week. Excuses related to Codabench evaluation delays will not be entertained.

7.4 Environment & Dependency Requirements

- Your submission must run correctly on a CPU-only environment.
- Even if you trained your agent/model using a GPU, the inference code must work on CPU without requiring CUDA or GPU support.

- If your agent uses trained weights, make sure to load them using CPU mapping (for example, `map_location="cpu"` when using PyTorch).
- Your code should not crash if a GPU is unavailable.
- Using PyTorch is allowed if required for your model.

8. Evaluation / Grading Criteria

Individual work only. Strict anti-cheating (code/viva checks → F if violation).

Your submission is evaluated on:

1. Understanding of environment and experimentation.
2. Submissions during development sub-phases (minimum 4 submissions per week).
3. Code quality, documentation, modularity.
4. Analysis of results (plots, why certain methods work/fail, visualizations like value maps or trajectories).
5. Novelty and creativity of the algorithm (new ideas, not just off-the-shelf).
6. Performance on the final phase (relative ranking).
7. Viva: defend choices, explain code, experiments.
8. Report: detail all attempts.
9. Minimum submissions per phase (penalty if missed).
10. Note: Authentic efforts (even if not fully successful) will be valued over leaderboard rank.

9. Submission Details: Report and Code

Submit via Google Form (details from TAs):

- **Report (LaTeX → PDF):** follow template (Abstract, Introduction/motivation, methods, experiments, analysis, conclusions).
    - The Report Template is provided here: https://drive.google.com/file/d/1PasQ4atuYoHdNQO33hjVzDQ6RdvclK3t/view?usp=sharing
    - The LLM declaration will be part of your report and its questionnaire will be shared in the template.
- **Code:** zip of your agent.py, any weights (.pth), notebooks/scripts for training/experiments. Naming:
    - `CS780-OBELIX-[yourfirstname]-[yourlastname]-[rollno]-report.pdf`

    - `…-code.zip`
- **Submission Form:** https://forms.gle/LrE4WBj7KPje7nX7A

10. Timeline

NOTE: Please do not rush for last-minute submissions, as Codabench takes around 15–20 minutes to evaluate each submission. Since many students will be participating, the evaluation time may increase due to higher load on Codabench. Therefore, it is advised not to wait until the last moment. Please try to make your submissions in the initial days of the week. Excuses related to Codabench evaluation delays will not be entertained.

10.1 Phase 1 (Level 1 Dev Phase)

- Start (Development Phase for Level-1): 9th March 2026, 12:00 AM IST
- End (Development Phase for Level-1): 22nd March 2026, 11:59 PM IST.

10.2 Phase 2 (Level 2 Dev Phase)

- Start (Development Phase for Level-2): 23rd March 2026, 12:00 AM IST
- End (Development Phase for Level-2): 29th March 2026, 11:59 PM IST.

10.3 Phase 3 (Level 3 Dev Phase)

- Start (Development Phase for Level-3): 30th March 2026, 12:00 AM IST
- End (Development Phase for Level-3): 5th April 2026, 11:59 PM IST.

10.4 Phase 4 (Test Phase)

- Final Submission Phase: 6th April 2026, 12:00 AM IST to 12th April 2026, 11:59 PM IST.
- Final submissions, report/code: 13th April 2026, 9:00 AM IST (submission via Google Form)
- Viva: 14th April 2026 to 24th April 2026.

Deadlines are strict — Codabench will automatically close as per the timeline.

11. Possible Ideas

11.1 Background Tutorials

To help you get started:

- Original paper (1991/1992): “Automatic Programming of Behaviour-based Robots using Reinforcement Learning” by Mahadevan & Connell — read for motivation and classic approach (Q-learning + clustering for generalization).
- CS780 Reinforcement Learning course lectures (especially Q-learning, SARSA, function approximation, policy gradients).
- Sutton & Barto book: Chapters on tabular RL, TD methods, eligibility traces.
- OpenAI Gym / simple RL tutorials for discrete action spaces.
- PyTorch RL examples: DQN for discrete actions.

You have learned several algorithms in the course, it would make sense to try out all those that are applicable in this setting. For example,

11.2 Tabular / Value-based Methods

Classic Q-learning or SARSA (like the original paper). Discretize state space (binary sensors already discrete). Use eligibility traces for faster credit assignment.

11.3 Function Approximation and Deep RL

DQN, Double DQN, Dueling DQN (with neural net on flattened obs). Or policy gradient methods like REINFORCE / PPO for direct policy learning. Great for generalization across seeds/difficulties.

11.4 Evolutionary Strategies & Heuristics

Simple hand-coded behaviors (e.g., Braitenberg-style or finite-state machine). Or CMA-ES / genetic algorithms to evolve policy parameters. Combine with RL (hierarchical or behavior-based).

11.5 What approach should I take?

No single best way — experiment heavily! Start with a random/heuristic baseline, then tabular Q, then deep methods. Try reward shaping, multi-task learning across difficulties, exploration tricks (epsilon-greedy, entropy bonus). Resource limits: keep nets small (Colab-friendly). Creativity wins — novel state representations or hybrid approaches encouraged.

NOTE: No matter what approach you develop, it MUST involve development of an RL-based algorithm.

12. Lost? Whom to contact?

Don't panic: RL on robots is hard but rewarding! Seek help instead of shortcuts. Use the Discord Channel: #capstone-project:

- Divyaksh Shukla : divyaksh@cse.iitk.ac.in
- Sanjeet Singh : sanjeet@cse.iitk.ac.in
- Dattatreya Pokala : pokala24@cse.iitk.ac.in
- Instructor: deeprl.iit@gmail.com

Ask about environment details, debugging, concepts — we want you to learn!

13. References

- Mahadevan, S., & Connell, J. (1992). Automatic programming of behavior-based robots using reinforcement learning. *Artificial Intelligence*, 55(2-3), 311–365. (https://cdn.aaai.org/AAAI/1991/AAAI91-120.pdf)
- *Reinforcement Learning: An Introduction* (https://web.stanford.edu/class/psych209/Readings/SuttonBartoIPRLBook2ndEd.pdf)